WEEK 3

SQL & EXCEL ASSIGNMENT

SQL & Excel for Data Analytics

Duration	1 Week
Total Marks	100
Submitted By	Nakul Mehra
College	Tula's Institute
Branch	MCA
Internship Organization	InternNova
Dataset	Sales Dataset

Practical Assignment Report

CONTENTS

Section	Description
Introduction	Overview of SQL and Excel skills covered in Week 3
Task 1	Introduction to Databases & SELECT
Task 2	WHERE, ORDER BY & Aggregate Functions
Task 3	GROUP BY & HAVING
Task 4	SQL Joins — INNER, LEFT & RIGHT
Task 5	SQL Subqueries
Task 6	Excel Formatting, Sorting & Filtering
Task 7	Conditional Formatting, IF, COUNTIF & SUMIF
Task 8	VLOOKUP & XLOOKUP
Task 9	Pivot Table & Charts
Key Findings	Important results from SQL and Excel analysis
Conclusion	Overall learning outcome
Submission Checklist	Files and links required for submission

INTRODUCTION

SQL and Microsoft Excel are widely used tools for data management, analysis and reporting. SQL provides a structured way to store, retrieve, filter, aggregate and combine data in relational databases, while Excel provides practical spreadsheet features for cleaning, sorting, filtering, formulas, lookups, PivotTables and charts.

This Week 3 assignment focuses on practical SQL operations using MySQL Workbench and spreadsheet-based analysis using Microsoft Excel. The SQL section uses an Employees table and a related Departments table. The Excel section uses a Sales Dataset containing 30 rows and 6 columns: Order_ID, Product, Category, Quantity, Price and Sales.

The assignment demonstrates a complete basic workflow: creating and querying database tables, applying conditions and sorting, calculating aggregate statistics, grouping records, joining related tables, using subqueries, formatting and filtering spreadsheet data, applying logical and conditional functions, performing lookups, and presenting category-wise sales using a PivotTable and PivotChart.

Tools Used

- MySQL Workbench — SQL database creation, queries and result analysis.
- Microsoft Excel — formatting, sorting, filtering, formulas, lookups, PivotTable and PivotChart.
- Sales Dataset — used for the Excel analysis section.

Assignment Outcome

The practical tasks build a foundation for database querying and spreadsheet-based data analysis, with emphasis on readable results and reusable analytical techniques.

TASK 1: INTRODUCTION TO DATABASES & SELECT

Objective: To understand the basic concept of a relational database and perform SELECT queries on a sample Employees table.

Brief Explanation: A database stores structured information in tables. In this task, an Employees table was created with employee ID, name, department, salary and city fields. SELECT statements were then used to retrieve complete records, selected columns and renamed columns using aliases.

Requirements Covered:

- Create and use a sample database/table.
- Retrieve all records using SELECT *.
- Retrieve specific columns.
- Use column aliases for clearer output.

SQL Source Code:

```
-- ======================================================
-- TASK 1: DATABASE & SELECT
-- ======================================================

CREATE DATABASE IF NOT EXISTS week3_sql;
USE week3_sql;

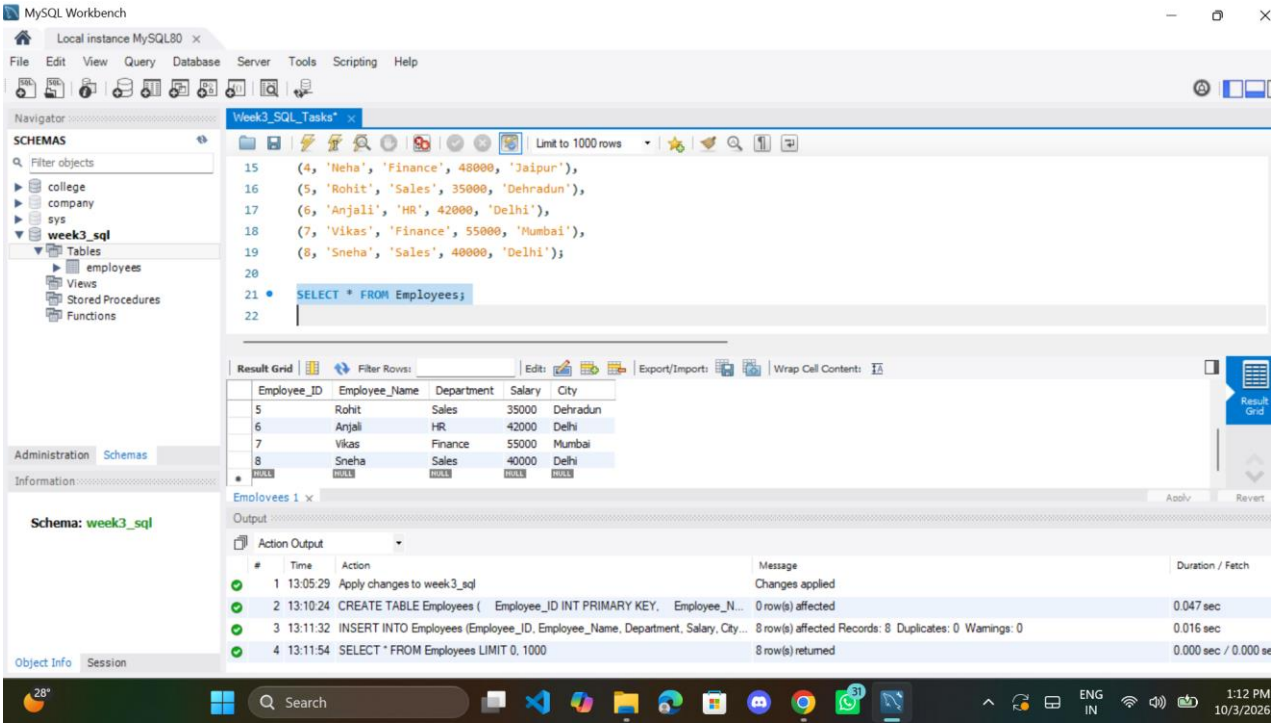
CREATE TABLE Employees (
    Employee_ID INT PRIMARY KEY,
    Employee_Name VARCHAR(50),
    Department VARCHAR(50),
    Salary INT,
    City VARCHAR(50)
);

INSERT INTO Employees
(Employee_ID, Employee_Name, Department, Salary, City)
VALUES
(1, 'Rahul', 'IT', 45000, 'Delhi'),
(2, 'Priya', 'HR', 38000, 'Dehradun'),
(3, 'Amit', 'IT', 52000, 'Delhi'),
(4, 'Neha', 'Finance', 48000, 'Jaipur'),
(5, 'Rohit', 'Sales', 35000, 'Dehradun'),
(6, 'Anjali', 'HR', 42000, 'Delhi'),
(7, 'Vikas', 'Finance', 55000, 'Mumbai'),
(8, 'Sneha', 'Sales', 40000, 'Delhi');

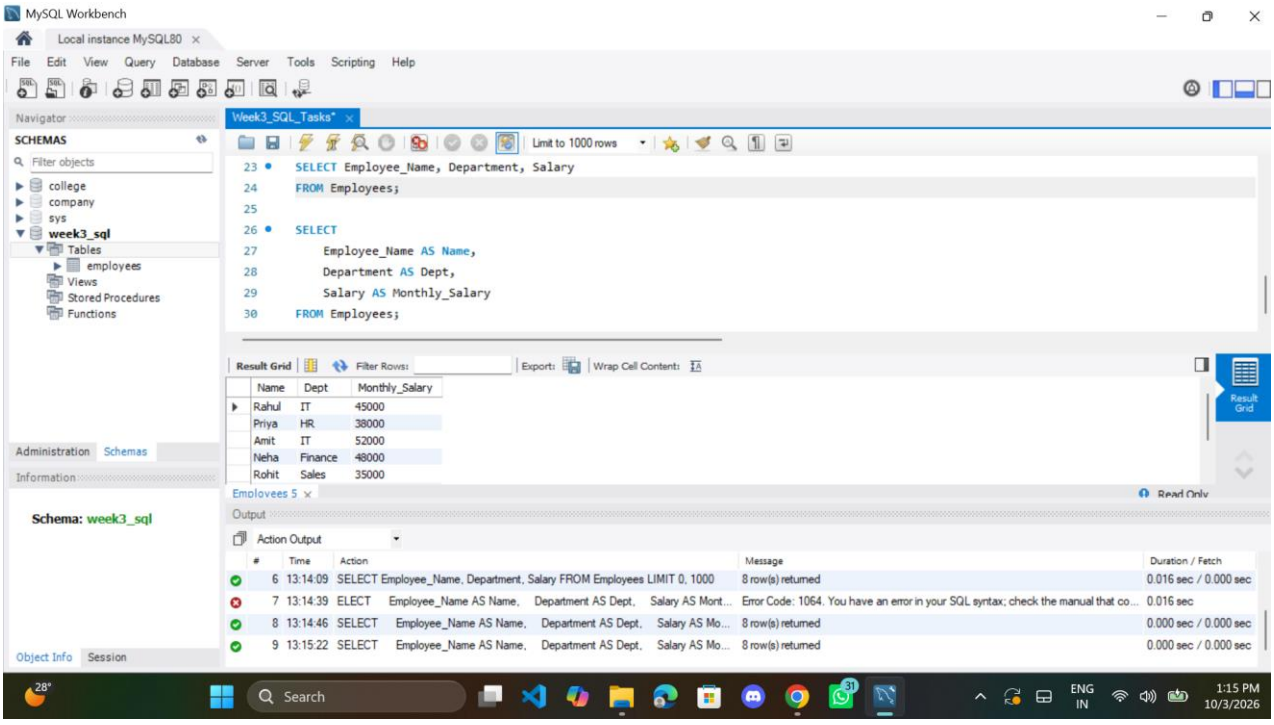
-- SELECT ALL RECORDS
SELECT * FROM Employees;

-- SELECT SPECIFIC COLUMNS
SELECT Employee_Name, Department, Salary
FROM Employees;

-- SELECT WITH ALIASES
SELECT
    Employee_Name AS Name,
    Department AS Dept,
    Salary AS Monthly_Salary
FROM Employees;
```



OUTPUT SCREENSHOT — Task 1: SELECT * from Employees



OUTPUT SCREENSHOT — Task 1: SELECT with column aliases

TASK 2: WHERE, ORDER BY & AGGREGATE FUNCTIONS

Objective: To filter records, sort results and calculate basic summary statistics using SQL aggregate functions.

Brief Explanation: WHERE filters rows according to a condition, ORDER BY sorts records, and aggregate functions summarize numerical data. COUNT(), SUM(), AVG(), MIN() and MAX() are demonstrated below.

SQL Source Code:

```
-- ================================================
-- TASK 2: WHERE, ORDER BY & AGGREGATE FUNCTIONS
-- ================================================

-- WHERE CLAUSE
-- Purpose: To filter employees whose salary is greater than 40000.
SELECT *
FROM Employees
WHERE Salary > 40000;

-- WHERE WITH COMPARISON
-- Purpose: To display only IT department employees.
SELECT Employee_Name, Department, Salary
FROM Employees
WHERE Department = 'IT';

-- ORDER BY DESC
-- Purpose: To sort employees from highest to lowest salary.
SELECT Employee_Name, Department, Salary
FROM Employees
ORDER BY Salary DESC;

-- ORDER BY ASC
-- Purpose: To sort employees from lowest to highest salary.
SELECT Employee_Name, Department, Salary
FROM Employees
ORDER BY Salary ASC;

-- COUNT()
-- Purpose: To count the total number of employees.
SELECT COUNT(*) AS Total_Employees
FROM Employees;

-- SUM()
-- Purpose: To calculate total salary.
SELECT SUM(Salary) AS Total_Salary
FROM Employees;

-- AVG()
-- Purpose: To calculate average salary.
SELECT AVG(Salary) AS Average_Salary
FROM Employees;

-- MIN()
-- Purpose: To find the minimum salary.
SELECT MIN(Salary) AS Minimum_Salary
FROM Employees;

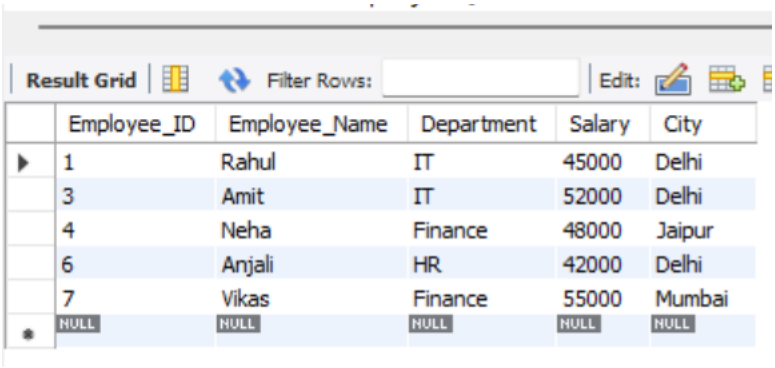
-- MAX()
-- Purpose: To find the maximum salary.
SELECT MAX(Salary) AS Maximum_Salary
FROM Employees;

-- ALL AGGREGATES
```

```sql
-- Purpose: To calculate all major summary statistics together.
SELECT
    COUNT(*) AS Total_Employees,
    SUM(Salary) AS Total_Salary,
    AVG(Salary) AS Average_Salary,
    MIN(Salary) AS Minimum_Salary,
    MAX(Salary) AS Maximum_Salary
FROM Employees;
```

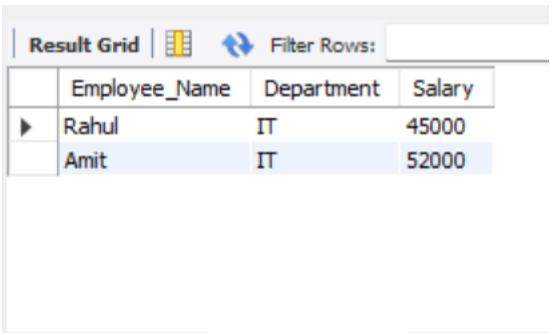
Expected / Recorded Aggregate Result:

Metric	Result
Total Employees	8
Total Salary	355000
Average Salary	44375
Minimum Salary	35000
Maximum Salary	55000



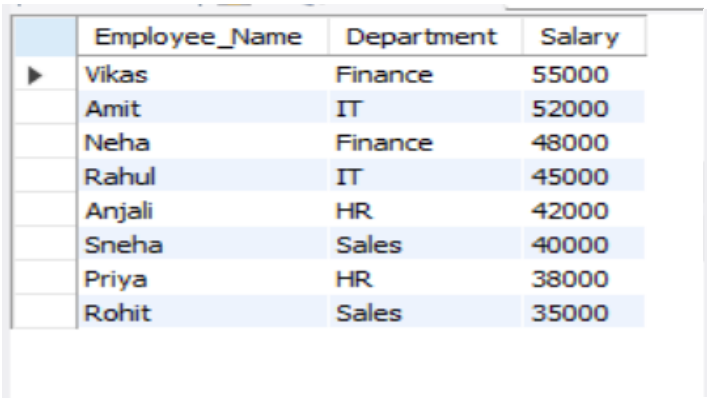
Result Grid | Filter Rows: | Edit:

Employee_ID	Employee_Name	Department	Salary	City
1	Rahul	IT	45000	Delhi
3	Amit	IT	52000	Delhi
4	Neha	Finance	48000	Jaipur
6	Anjali	HR	42000	Delhi
7	Vikas	Finance	55000	Mumbai
NULL	NULL	NULL	NULL	NULL

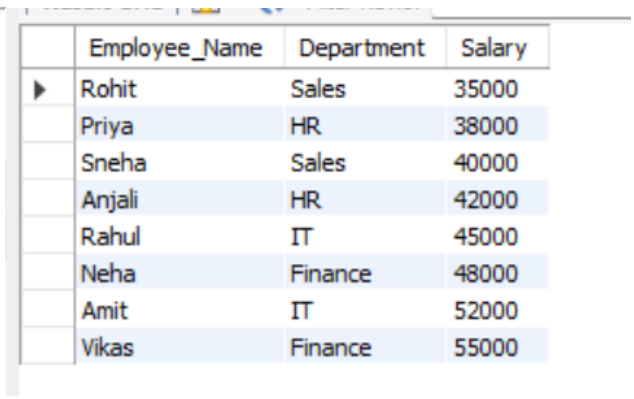


Result Grid | Filter Rows:

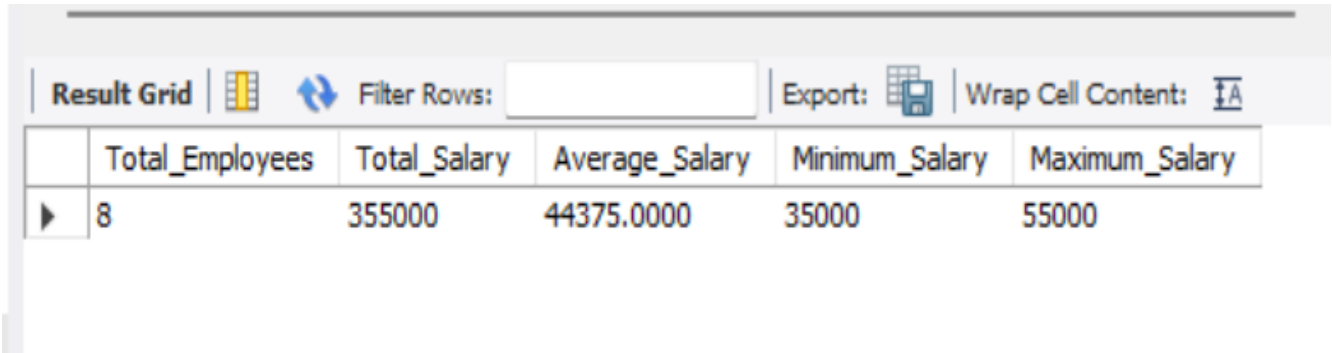
Employee_Name	Department	Salary
Rahul	IT	45000
Amit	IT	52000



Employee_Name	Department	Salary
Vikas	Finance	55000
Amit	IT	52000
Neha	Finance	48000
Rahul	IT	45000
Anjali	HR	42000
Sneha	Sales	40000
Priya	HR	38000
Rohit	Sales	35000



Employee_Name	Department	Salary
Rohit	Sales	35000
Priya	HR	38000
Sneha	Sales	40000
Anjali	HR	42000
Rahul	IT	45000
Neha	Finance	48000
Amit	IT	52000
Vikas	Finance	55000



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Total_Employees	Total_Salary	Average_Salary	Minimum_Salary	Maximum_Salary
8	355000	44375.0000	35000	55000

TASK 3: GROUP BY & HAVING

Objective: To summarize employee information department-wise and filter grouped results using HAVING.

Brief Explanation: GROUP BY creates groups based on a column and allows aggregate functions to be applied to each group. HAVING filters grouped results after aggregation.

SQL Source Code:

```
-- ====================================================
-- TASK 3: GROUP BY & HAVING
-- ====================================================

-- GROUP BY WITH COUNT
-- Purpose: To calculate the number of employees in each department.
SELECT
    Department,
    COUNT(*) AS Total_Employees
FROM Employees
GROUP BY Department;

-- GROUP BY WITH SUM
-- Purpose: To calculate total salary for each department.
SELECT
    Department,
    SUM(Salary) AS Total_Salary
FROM Employees
GROUP BY Department;

-- GROUP BY WITH AVG
-- Purpose: To calculate average salary for each department.
SELECT
    Department,
    AVG(Salary) AS Average_Salary
FROM Employees
GROUP BY Department;

-- MULTIPLE AGGREGATES
-- Purpose: To calculate employee count, total salary and average salary.
SELECT
    Department,
    COUNT(*) AS Total_Employees,
    SUM(Salary) AS Total_Salary,
    AVG(Salary) AS Average_Salary
FROM Employees
GROUP BY Department;

-- HAVING
-- Purpose: To display departments whose total salary is greater than 80000.
SELECT
    Department,
    SUM(Salary) AS Total_Salary
FROM Employees
GROUP BY Department
HAVING SUM(Salary) > 80000;

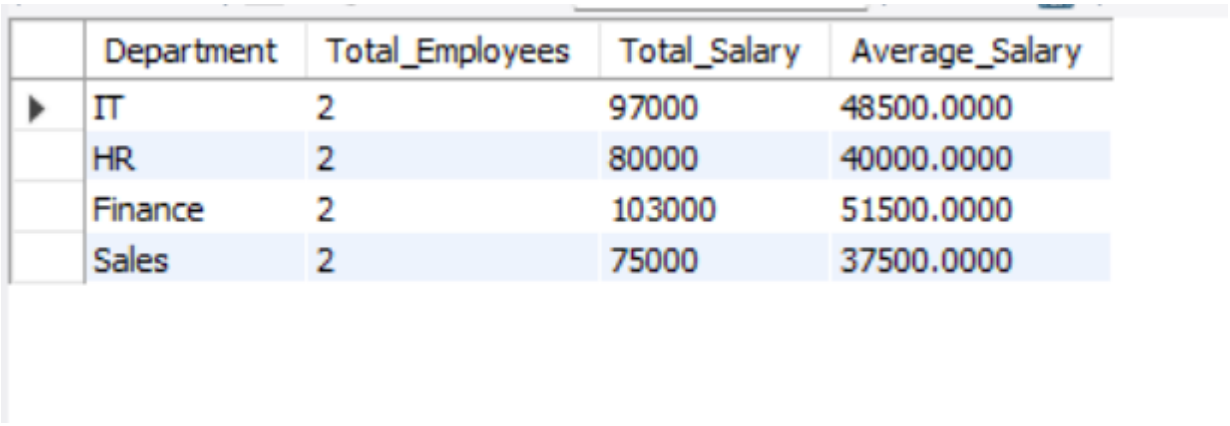
-- HAVING WITH COUNT
-- Purpose: To display departments having more than one employee.
SELECT
    Department,
    COUNT(*) AS Total_Employees
FROM Employees
```

```
GROUP BY Department
HAVING COUNT(*) > 1;
```

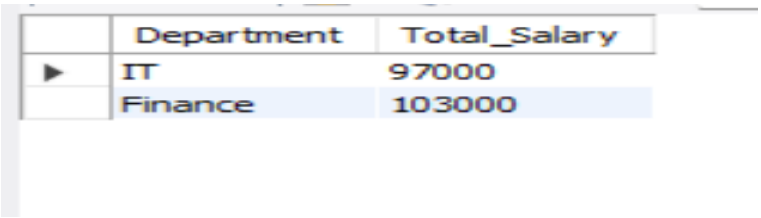
Department	Employees	Total Salary	Average Salary
IT	2	97000	48500
HR	2	80000	40000
Finance	2	103000	51500
Sales	2	75000	37500

HAVING Result

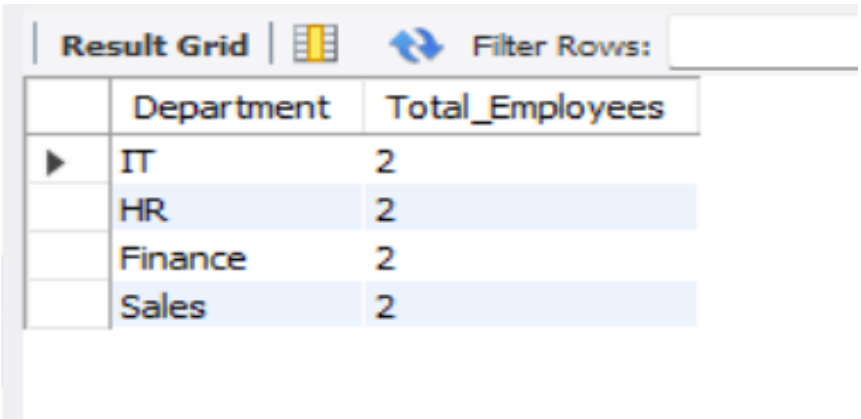
Departments with total salary above 80000 are IT (97000) and Finance (103000). All four departments have more than one employee.



Department	Total_Employees	Total_Salary	Average_Salary
IT	2	97000	48500.0000
HR	2	80000	40000.0000
Finance	2	103000	51500.0000
Sales	2	75000	37500.0000



Department	Total_Salary
IT	97000
Finance	103000



Department	Total_Employees
IT	2
HR	2
Finance	2
Sales	2

TASK 4: SQL JOINS — INNER, LEFT & RIGHT

Objective: To combine data from related tables and understand the difference between INNER JOIN, LEFT JOIN and RIGHT JOIN.

Brief Explanation: A JOIN combines rows from two tables using a related column. The Departments table contains IT, HR, Finance, Sales and Marketing. Marketing has no matching employee, which demonstrates the behavior of a RIGHT JOIN.

SQL Source Code:

```
-- ======================================================
-- TASK 4: SQL JOINS
-- ======================================================

-- CREATE DEPARTMENTS TABLE
-- Purpose: To create a related table for department details.
CREATE TABLE Departments (
    Department_ID INT PRIMARY KEY,
    Department_Name VARCHAR(50),
    Location VARCHAR(50)
);

-- INSERT DEPARTMENT DATA
-- Purpose: To add department information.
INSERT INTO Departments
(Department_ID, Department_Name, Location)
VALUES
(101, 'IT', 'Delhi'),
(102, 'HR', 'Dehradun'),
(103, 'Finance', 'Jaipur'),
(104, 'Sales', 'Mumbai'),
(105, 'Marketing', 'Delhi');

-- INNER JOIN
-- Purpose: To display employees with matching departments.
SELECT
    Employees.Employee_Name,
    Employees.Department,
    Departments.Location
FROM Employees
INNER JOIN Departments
ON Employees.Department = Departments.Department_Name;

-- LEFT JOIN
-- Purpose: To display all employees and matching department information.
SELECT
    Employees.Employee_Name,
    Employees.Department,
    Departments.Location
FROM Employees
LEFT JOIN Departments
ON Employees.Department = Departments.Department_Name;

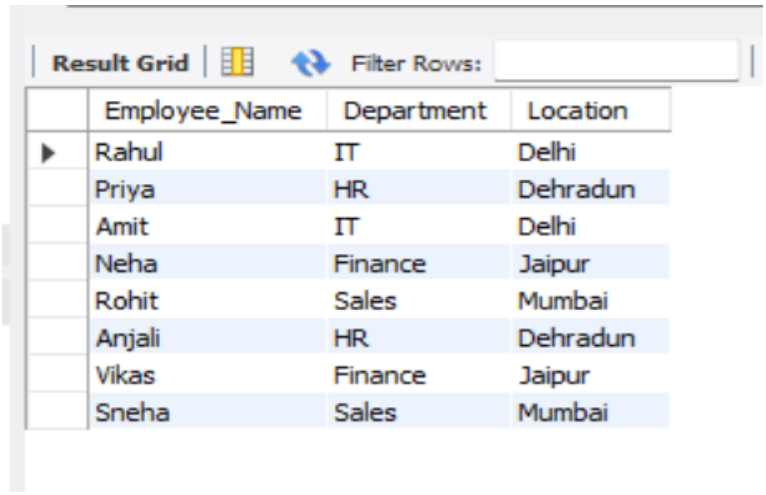
-- RIGHT JOIN
-- Purpose: To display all departments, including departments without employees.
SELECT
    Employees.Employee_Name,
    Employees.Department,
    Departments.Location
```

```
FROM Employees
RIGHT JOIN Departments
ON Employees.Department = Departments.Department_Name;
```

Join	Main Result
INNER JOIN	Matching employee-department records are returned.
LEFT JOIN	All 8 employees are retained; matching department data is shown.
RIGHT JOIN	All 5 departments are retained; Marketing appears with NULL employee fields.

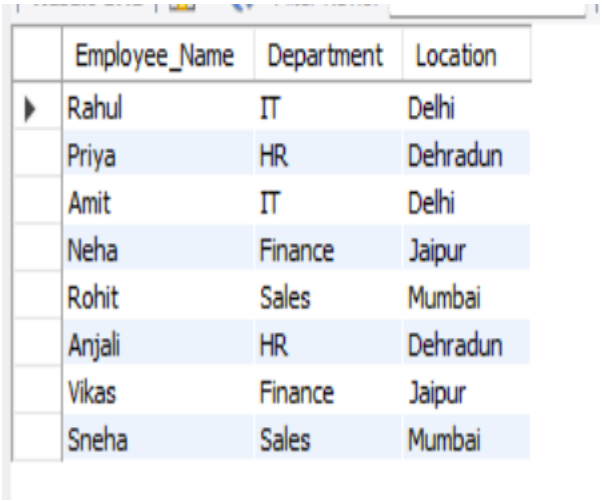
Key Observation

RIGHT JOIN clearly demonstrates an unmatched department: Marketing has no employees, so employee-related fields are NULL.

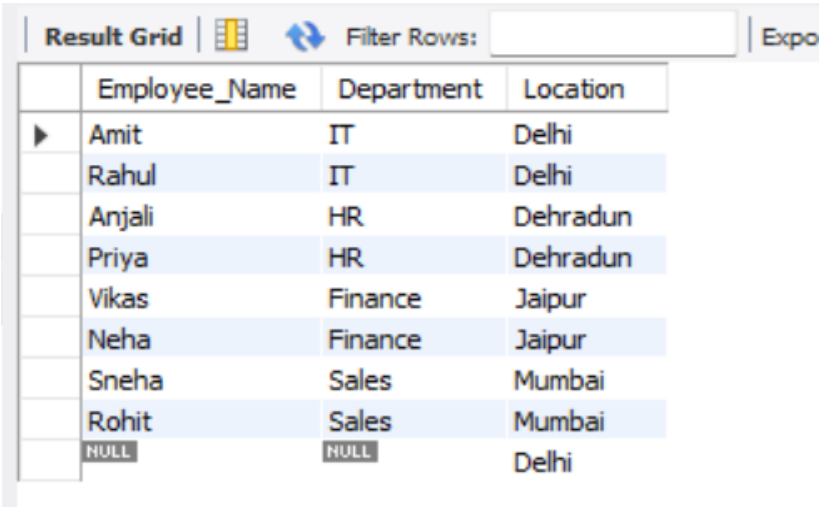


Result Grid | Filter Rows:

Employee_Name	Department	Location
Rahul	IT	Delhi
Priya	HR	Dehradun
Amit	IT	Delhi
Neha	Finance	Jaipur
Rohit	Sales	Mumbai
Anjali	HR	Dehradun
Vikas	Finance	Jaipur
Sneha	Sales	Mumbai



Employee_Name	Department	Location
Rahul	IT	Delhi
Priya	HR	Dehradun
Amit	IT	Delhi
Neha	Finance	Jaipur
Rohit	Sales	Mumbai
Anjali	HR	Dehradun
Vikas	Finance	Jaipur
Sneha	Sales	Mumbai



Result Grid | Filter Rows: | Expo

Employee_Name	Department	Location
Amit	IT	Delhi
Rahul	IT	Delhi
Anjali	HR	Dehradun
Priya	HR	Dehradun
Vikas	Finance	Jaipur
Neha	Finance	Jaipur
Sneha	Sales	Mumbai
Rohit	Sales	Mumbai
NULL	NULL	Delhi

TASK 5: SQL SUBQUERIES

Objective: To use nested SELECT statements to compare employee records against calculated or grouped values.

Brief Explanation: A subquery is a query placed inside another query. The examples below use AVG(), MAX(), GROUP BY and HAVING inside subqueries.

SQL Source Code:

```
-- ======================================================
-- TASK 5: SQL SUBQUERIES
-- ======================================================

-- 1. ABOVE AVERAGE SALARY
-- Purpose: To find employees earning more than the average salary.
SELECT
    Employee_Name,
    Department,
    Salary
FROM Employees
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Employees
);

-- 2. MAXIMUM SALARY
-- Purpose: To find employee(s) having the highest salary.
SELECT
    Employee_Name,
    Department,
    Salary
FROM Employees
WHERE Salary = (
    SELECT MAX(Salary)
    FROM Employees
);

-- 3. DEPARTMENTS WITH MORE THAN ONE EMPLOYEE
-- Purpose: To find employees belonging to departments with more than one employee.
SELECT
    Employee_Name,
    Department,
    Salary
FROM Employees
WHERE Department IN (
    SELECT Department
    FROM Employees
    GROUP BY Department
    HAVING COUNT(*) > 1
);
```

Subquery	Result
Salary > average salary	Amit (52000), Neha (48000), Vikas (55000)
Salary = maximum salary	Vikas (55000)
Department has >1 employee	All 8 employees, because each department has 2 employees

Result Grid | Filter Rows:

Employee_Name	Department	Salary
Rahul	IT	45000
Amit	IT	52000
Neha	Finance	48000
Vikas	Finance	55000

Result Grid | Filter Rows:

Employee_Name	Department	Salary
Vikas	Finance	55000

Result Grid | Filter Rows:

Employee_Name	Department	Salary
Rahul	IT	45000
Priya	HR	38000
Amit	IT	52000
Neha	Finance	48000
Rohit	Sales	35000
Anjali	HR	42000
Vikas	Finance	55000
Sneha	Sales	40000

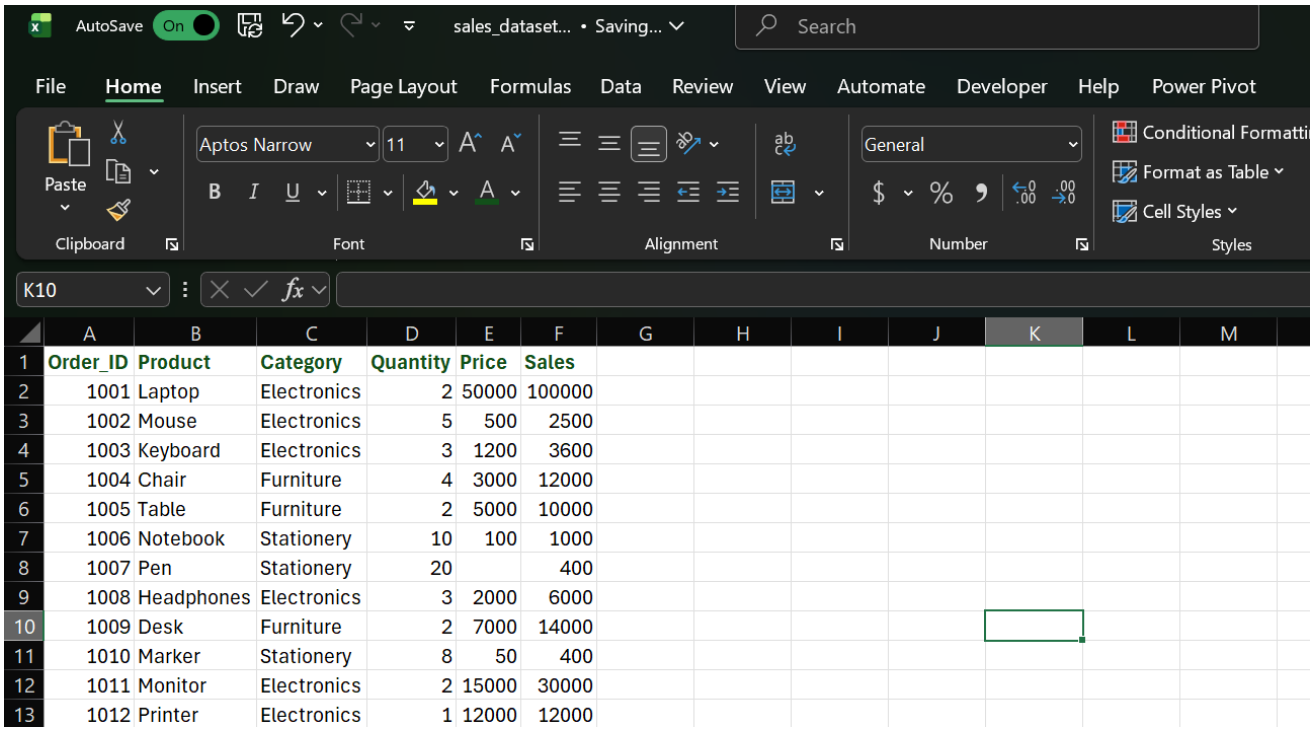
TASK 6: EXCEL FORMATTING, SORTING & FILTERING

Objective: To prepare the Sales Dataset for analysis by improving readability and using sorting and filtering features.

Brief Explanation: The sales dataset contains Order_ID, Product, Category, Quantity, Price and Sales. The header row was formatted for readability, columns were adjusted, Sales was sorted from highest to lowest, and Excel filtering was used to view category-specific records.

Task Requirements Covered:

- Format the dataset header and adjust column widths.
- Sort Sales from highest to lowest.
- Apply a filter to inspect selected categories.



	A	B	C	D	E	F
1	Order_ID	Product	Category	Quantity	Price	Sales
2	1001	Laptop	Electronics	2	50000	100000
3	1002	Mouse	Electronics	5	500	2500
4	1003	Keyboard	Electronics	3	1200	3600
5	1004	Chair	Furniture	4	3000	12000
6	1005	Table	Furniture	2	5000	10000
7	1006	Notebook	Stationery	10	100	1000
8	1007	Pen	Stationery	20		400
9	1008	Headphones	Electronics	3	2000	6000
10	1009	Desk	Furniture	2	7000	14000
11	1010	Marker	Stationery	8	50	400
12	1011	Monitor	Electronics	2	15000	30000
13	1012	Printer	Electronics	1	12000	12000

OUTPUT SCREENSHOT — Task 6: Formatted Sales Dataset

Excel Result: The dataset is arranged in a readable table structure and the Sales column can be sorted/filtered for analysis.

TASK 7: CONDITIONAL FORMATTING, IF, COUNTIF & SUMIF

Objective: To highlight important sales values and create formula-based analytical indicators.

Brief Explanation: Conditional Formatting was applied to highlight Sales values greater than 10,000. The IF function classified records as High Sales or Normal Sales. COUNTIF counted Electronics records, while SUMIF calculated total Electronics sales.

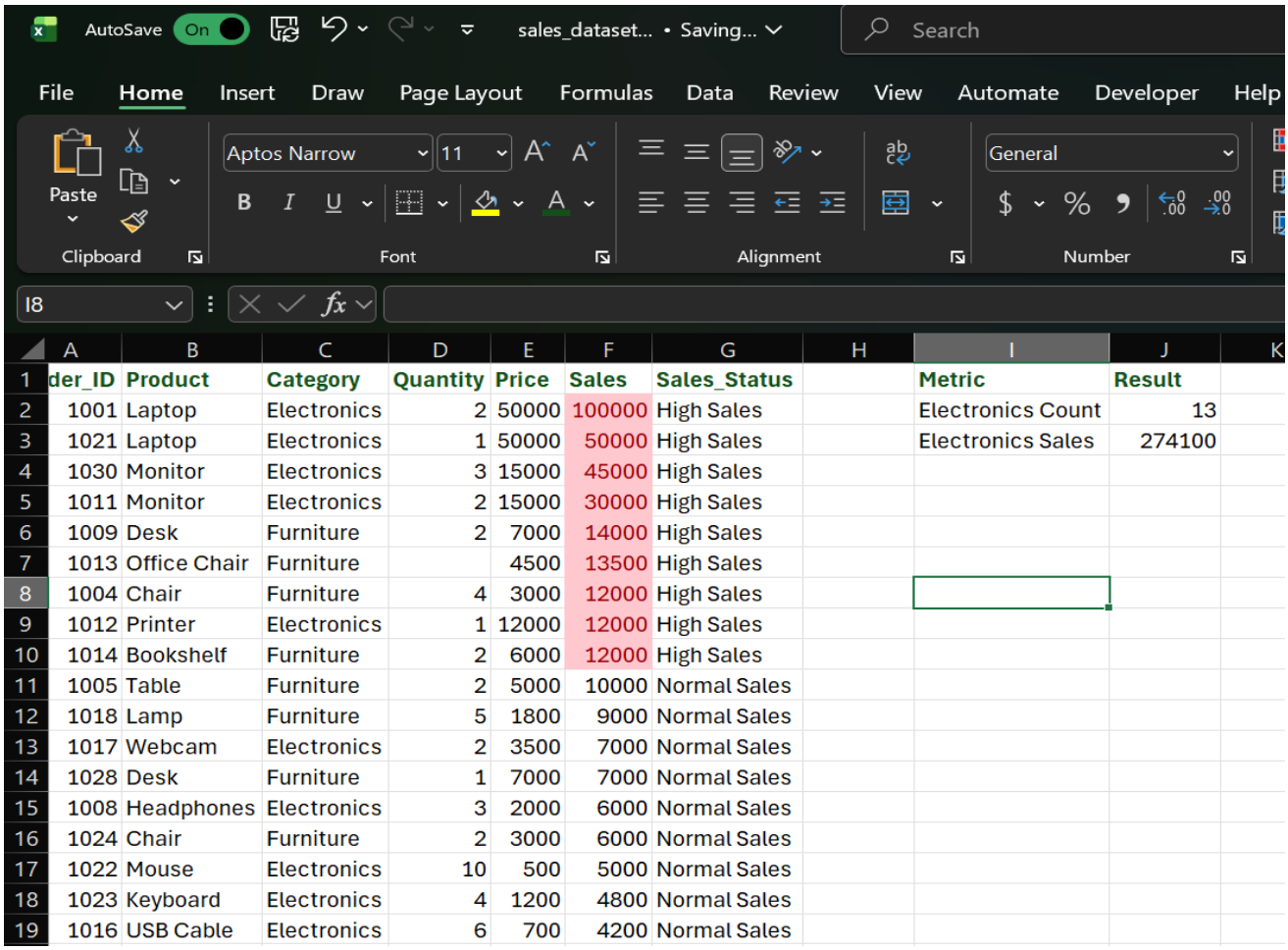
Formulas Used:

```
-- IF formula in G2
=IF(F2>10000,"High Sales","Normal Sales")

-- COUNTIF
=COUNTIF(C:C,"Electronics")

-- SUMIF
=SUMIF(C:C,"Electronics",F:F)
```

Metric	Formula	Recorded Result
Electronics Count	=COUNTIF(C:C,"Electronics")	13
Electronics Sales	=SUMIF(C:C,"Electronics",F:F)	274100



	A	B	C	D	E	F	G	H	I	J	K
1	der_ID	Product	Category	Quantity	Price	Sales	Sales_Status		Metric	Result	
2	1001	Laptop	Electronics	2	50000	100000	High Sales		Electronics Count	13	
3	1021	Laptop	Electronics	1	50000	50000	High Sales		Electronics Sales	274100	
4	1030	Monitor	Electronics	3	15000	45000	High Sales				
5	1011	Monitor	Electronics	2	15000	30000	High Sales				
6	1009	Desk	Furniture	2	7000	14000	High Sales				
7	1013	Office Chair	Furniture		4500	13500	High Sales				
8	1004	Chair	Furniture	4	3000	12000	High Sales				
9	1012	Printer	Electronics	1	12000	12000	High Sales				
10	1014	Bookshelf	Furniture	2	6000	12000	High Sales				
11	1005	Table	Furniture	2	5000	10000	Normal Sales				
12	1018	Lamp	Furniture	5	1800	9000	Normal Sales				
13	1017	Webcam	Electronics	2	3500	7000	Normal Sales				
14	1028	Desk	Furniture	1	7000	7000	Normal Sales				
15	1008	Headphones	Electronics	3	2000	6000	Normal Sales				
16	1024	Chair	Furniture	2	3000	6000	Normal Sales				
17	1022	Mouse	Electronics	10	500	5000	Normal Sales				
18	1023	Keyboard	Electronics	4	1200	4800	Normal Sales				
19	1016	USB Cable	Electronics	6	700	4200	Normal Sales				

OUTPUT SCREENSHOT — Task 7: Conditional Formatting and Formula Results

TASK 8: VLOOKUP & XLOOKUP

Objective: To retrieve category information for products using Excel lookup functions.

Brief Explanation: A lookup table was created in columns M:N containing Product and Category. VLOOKUP retrieves the category using a column index, while XLOOKUP searches the product column and returns the corresponding category.

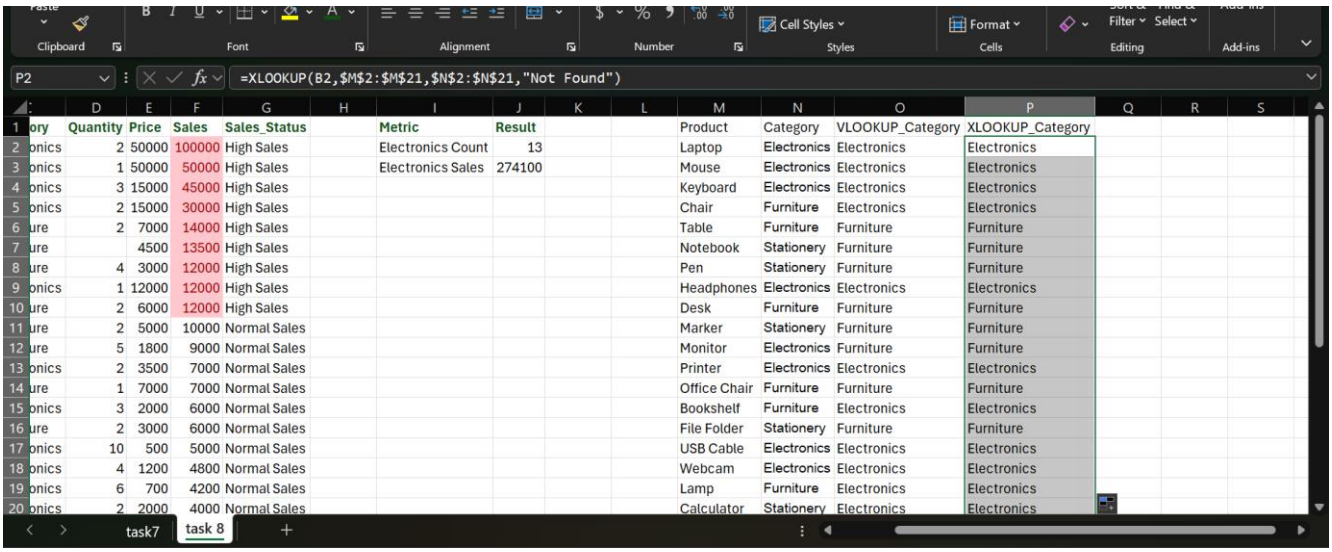
Lookup Table:

Product	Category
Laptop	Electronics
Mouse	Electronics
Keyboard	Electronics
Chair	Furniture
Table	Furniture
Notebook	Stationery
Pen	Stationery
Headphones	Electronics
Desk	Furniture
Marker	Stationery
Monitor	Electronics
Printer	Electronics
Office Chair	Furniture
Bookshelf	Furniture
File Folder	Stationery
USB Cable	Electronics
Webcam	Electronics
Lamp	Furniture
Calculator	Stationery
Stapler	Stationery

Formulas Used:

```
-- VLOOKUP in O2
=VLOOKUP(B2,$M$2:$N$21,2,FALSE)

-- XLOOKUP in P2
=XLOOKUP(B2,$M$2:$M$21,$N$2:$N$21,"Not Found")
```



OUTPUT SCREENSHOT — Task 8: VLOOKUP and XLOOKUP Results

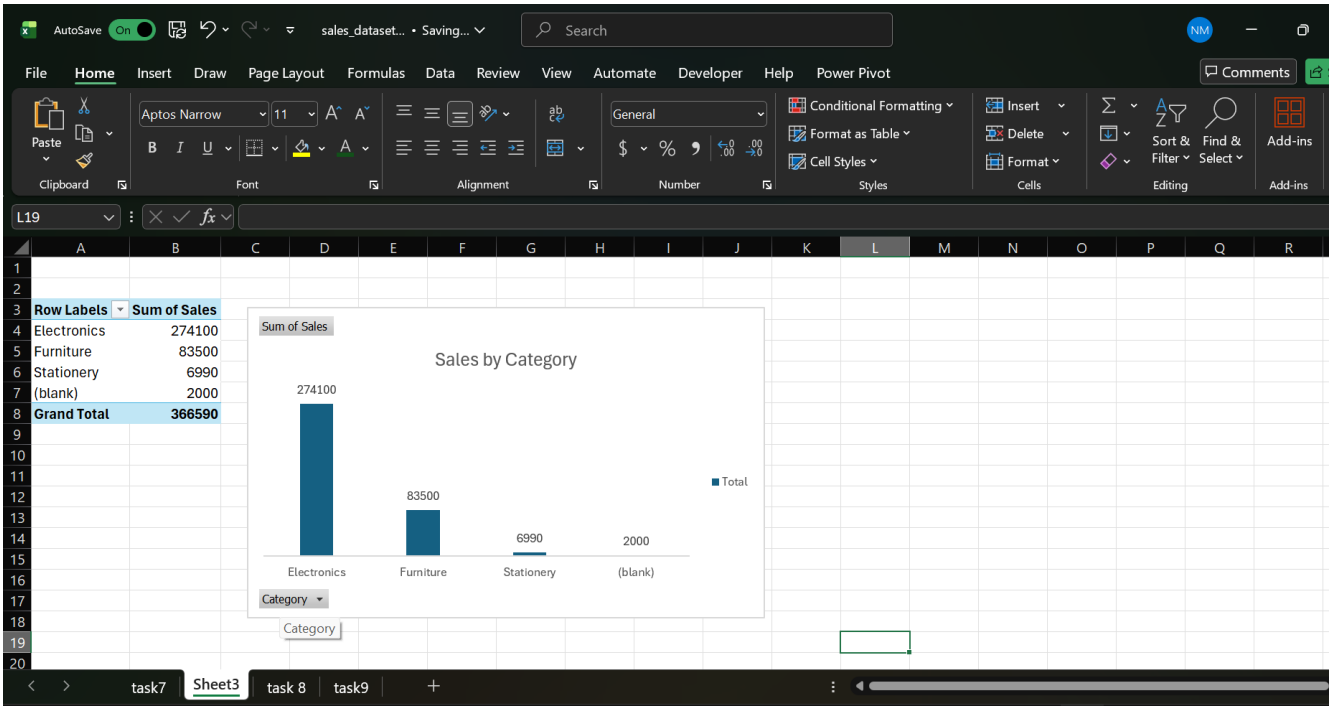
TASK 9: PIVOT TABLE & CHARTS

Objective: To summarize total sales by product category and visualize the result using a PivotChart.

Brief Explanation: A PivotTable was created from the sales dataset with Category in Rows and Sales in Values. A clustered column PivotChart titled “Sales by Category” was then created from the PivotTable.

PivotTable Result:

Category	Sum of Sales
Electronics	274100
Furniture	83500
Stationery	6990
(blank)	2000
Grand Total	366590



OUTPUT SCREENSHOT — Task 9: PivotTable and Sales by Category Chart

Chart Interpretation

Electronics contributes the highest sales (274100), followed by Furniture (83500). Stationery contributes 6990. The blank category accounts for 2000 because the source dataset contains a missing Category value.

KEY FINDINGS / INSIGHTS

The following observations were obtained from the SQL and Excel work completed in this assignment:

- The Employees database contains 8 employee records distributed across IT, HR, Finance and Sales.
- Total employee salary is 355000, with an average salary of 44375.
- Vikas has the highest salary at 55000, while Rohit has the lowest salary at 35000.
- Finance has the highest department-level salary total at 103000.
- The RIGHT JOIN identifies Marketing as an unmatched department with no employee records.
- Subquery analysis identifies three employees earning above the overall average salary.
- Excel analysis identified 13 Electronics records and Electronics sales of 274100.
- The PivotTable shows Electronics as the highest-selling category with 274100 in sales.
- The overall PivotTable sales total is 366590, including 2000 from records with a blank Category.
- The VLOOKUP and XLOOKUP exercises successfully returned category information for the product lookup table.

CONCLUSION

This Week 3 SQL & Excel assignment provided practical experience in two important areas of Data Analytics: relational database querying and spreadsheet-based analysis. SQL tasks covered database creation, SELECT statements, filtering, sorting, aggregate functions, GROUP BY, HAVING, joins and subqueries.

The Excel tasks extended the analysis workflow by introducing professional formatting, sorting, filtering, conditional formatting, logical and conditional functions, lookup functions, PivotTables and charts. The final PivotTable and chart provided a clear visual summary of category-wise sales.

Overall, the assignment strengthened the ability to retrieve, transform, summarize and communicate data using SQL and Excel, which are essential skills for entry-level Data Analytics work.

SUBMISSION CHECKLIST

Item	Status / Action
PDF Report	Generate/export this DOCX as PDF if the portal specifically requires PDF.
SQL File	Week3_SQL_Tasks.sql
Excel Workbook	sales_dataset.xlsx
SQL Screenshots	Include MySQL Workbench screenshots for Tasks 1–5 as required.
Excel Screenshots	Tasks 6–9 screenshots are included in this report.
Google Drive	Upload PDF/report, SQL file and Excel workbook into one Week 3 folder.
Drive Permission	Set sharing to Anyone with the link → Viewer, if required by the instructor.
Submission Form	Provide the Google Drive/cloud link and upload the final PDF/required file.

GOOGLE DRIVE LINK

https://drive.google.com/drive/folders/1Y0B0ejt4Esd2Vl6RLG-QM1Ugmmv7U5ud?usp=sharing

Sharing Permission: Anyone with the link — Viewer (or as required by instructor).

Recommended Week 3 Drive Folder Contents:

- Week_3_SQL_Excel_Assignment_Report.docx / PDF
- Week3_SQL_Tasks.sql
- sales_dataset.xlsx

Final Reminder: Open the shared Drive link in an incognito/private browser window before submission to verify that the files are accessible.

END OF REPORT